课程笔记：面向编程与软件开发的 agent

Bowen-mantou（整理）

2026-10-23

视频作者/频道：B 站「常青藤名校课」· CMU 11-768 AI Agents（英文原声字幕版）
发布日期：2026-09-09
视频时长：78:49
视频链接：https://www.bilibili.com/video/BV1a6Yx62EH4?p=20
PDF 制作 Skill：https://github.com/wdkns/wdkns-skills

目录

1 引言：会写代码的模型，与会改代码库的 agent **2**

1.1 代码生成的两类任务 . . . 2
1.2 本讲路线图 . . . 2
1.3 本章小结 . . . 2

2 预训练：数据、tokenization 与长上下文 **2**

2.1 预训练要学到什么 . . . 2
2.2 Tokenization：为代码而改的分词器 . . . 3
2.3 长上下文：把「一个仓库」放进窗口 . . . 3
2.4 本章小结 . . . 3

3 中训练：把通用模型改造成代码模型 **3**

3.1 代码配比：不能只喂代码 . . . 4
3.2 Infilling：让模型学会「填中间的空」 . . . 4
3.3 更多代码数据源：从仓库里「免费」挖数据 . . . 4
3.4 本章小结 . . . 4

4 代码评测：从表层相似到执行判定 **5**

4.1 和参考解比较：BLEU、CodeBLEU 与 CodeBERTScore . . . 5
4.2 执行判定：跑测试 . . . 5
4.3 单步代码生成的主流 benchmark . . . 5
4.4 非功能需求：正确之外的维度 . . . 6
4.5 本章小结 . . . 6

5 单步代码模型的 RL：从测试结果学习推理 **6**

5.1 本章小结 . . . 6

6 Agentic coding：localize–edit–verify 循环 **7**

6.1 三步循环 . . . 7
6.2 一个反例：Agentless 固定工作流 . . . 7
6.3 只给一个 bash 工具行不行? . . . 7
6.4 编辑格式：diff vs 搜索替换 . . . 7
6.5 代码定位：从 grep 到依赖图 . . . 8
6.6 本章小结 . . . 8

7 评测与训练数据工程：SWE-bench 及其生态 **8**

7.1 SWE-bench 是怎么构造的 . . . 8
7.2 评测是改进 agent 的最大瓶颈 . . . 9
7.3 多 harness 训练 . . . 9
7.4 本章小结 . . . 9

8 不只是修 bug：前端、多语言与更广义的软件开发 **10**
8.1 前端开发：需要浏览器验证 10
8.2 内循环与外循环 10
8.3 任务谱系：修复之外的软件工程 10
8.4 本章小结 11

9 代码世界模型与训练迁移 **11**
9.1 预测执行：让模型「在脑子里运行程序」 11
9.2 训练能迁移到新的开发任务 11
9.3 本章小结 11

10 总结与延伸 **11**
10.1 主讲人的收尾 11
10.2 把本讲压缩成六句话 12
10.3 延伸思考 12
10.4 拓展阅读（课程中提到的文献，供进一步查阅） 13

1 引言：会写代码的模型，与会改代码库的 agent

本讲的主题是「面向编程与软件开发的 agent」。主讲人一上来先纠正了标题可能带来的误解：首先要谈的其实**不是 agent，而是一个会写代码的语言模型**——它是所有编程 agent 的地基。也就是说，这一讲的主线是：先把「代码模型」这件事讲清楚（预训练、中训练、评测、RL），再把它装进 agent 的循环里（工具、编辑格式、仓库级任务），最后扩展到前端、多语言与更广义的软件开发。

1.1 代码生成的两类任务

图 **??** 把「会写代码」分成两类：第一类是**写代码**（writes code，single completion）——给一段自然语言描述，模型一次性生成一段程序，比如「把这份 CSV 画成柱状图」；第二类是**修改仓库**（modifies repositories，modify multiple files）——在既有代码库里修一个 issue、加一个 feature，往往要改多个文件。第三类其实还有：软件开发远不止写代码和修 bug，还有很多别的工作，这一讲后半段会讲。

和所有 agent 一样，编程 agent 也由三部分构成：**一个擅长写代码的大模型、一组工具、上下文/提示管理**。上一讲已经详细讲了长上下文、记忆与技能，所以本讲把重点放在前两块：模型本身怎么来的，以及工具（尤其是编辑工具与 harness）怎么影响能力。

1.2 本讲路线图

本讲路线图

代码模型：预训练（数据清洗、tokenization、长上下文）→ 中训练（代码配比、infilling、更多代码数据源）→ 评测（表层相似 vs 执行判定）→ 单步代码模型的 RL（从测试结果学习推理）→ agentic coding（localize–edit–verify 循环、shell 工具、编辑格式、代码定位）→ 评测与训练数据（SWE-bench 生态、注入 bug、多 harness 训练）→ 前端与其他软件工程任务（inner/outer loop）→ 代码世界模型与训练迁移。

1.3 本章小结

编程 agent 的地基是一个会写代码的模型；代码任务分为「一次性生成」与「修改多文件仓库」两类，后者需要工具与循环。本讲先讲代码模型本身（预训练、中训练、评测、RL），再讲 agentic 工具与仓库级任务，最后扩展到前端与更广义的软件开发。

2 预训练：数据、tokenization 与长上下文

2.1 预训练要学到什么

代码模型的预训练本质上还是 next-token prediction，但它和「只回答问题的模型」有几个关键差别，主讲人从数据与 token 两侧展开。

数据混合。代码模型不能只喂代码：它还需要网页文本（理解用户的语言）、技术文章（解释代码怎么工作）、数学（提升逻辑推理）、源代码与文档。也就是说，代码数据要**混**进通用语料，而不是替换通用语料。

代码的数据分布很特殊。主讲人举了一个经典例子：早期预处理会「归一化空白」，把所有连续空格压成一个——这在英文文本里没问题，但在 Python 里**空白是有语义的**（缩进决定代码块），一

压就全错了。类似地还要注意文件边界怎么表达、markdown 代码块要保持在代码块里、代码与解释文本的对应关系不能被破坏。

清洗与去重比文本模型更难。需要按语言与来源过滤；去掉自动生成的、编码错误的、低价值重复的代码；尤其要**去重 forks、vendor 库、近乎相同的文件、抄来抄去的题解**——GitHub 上 fork 数量巨大，不处理会把训练集质量整体拉低。此外还要处理许可证与来源状态（proprietary 代码、opt-out、只要新代码）、以及敏感数据过滤（很多人把 API key 上传到仓库，业界常用 TruffleHog 一类工具扫描），并据此构造训练集与测试集。

图 **??** 是 StarCoder 系列的一个经典结论：模型规模固定时，**大多数编程语言的 pass@1 都随着训练 token 数上升而提高**——数据量对代码能力几乎总是有效的。幻灯片里的配置是 StarCoder-Base（15.5B 参数、1T token、80+ 语言、来源包括源码/issue/commit/notebook）。

2.2 Tokenization：为代码而改的分词器

把代码切成 token 看起来是细节，实际上直接影响模型能力。图 **??** 给了三条原则：用 **byte-level BPE** 在代码与文本上学习 **code-aware 的子词**（标识符可以跨 token）；**保留空白**（空格、tab、换行既携带语法也携带字面内容）；**压缩高频片段**（把常见的连续空白合并成一个 token，而不是一个空格一个 token）。

幻灯片底部给了一个真实例子：`if ready:\n return value` 被切成 `if` / ·`ready` / `:` / `\n` ··· / ·`return` / ·`value` 这样的片段——注意换行加四个空格被压成了一个 token。主讲人还解释了一个工程取舍：理论上 BPE 可以完全按频率合并，但实践中通常把空白与非空白分开处理，避免模型输出「半个单词」这种奇怪行为；不过也要小心过度压缩——比如 `import numpy as np` 如果被合并成一个 token，自动补全这类任务就会出问题。

2.3 长上下文：把「一个仓库」放进窗口

代码模型特别需要长上下文，因为你希望模型能读到整个仓库里的相关信息。但直接拼接文件并不可行：把所有文件拼起来，通常会远超最长上下文（主讲人提到训练上限常见是 100 万 token，而很多代码库比这更大，比如据称有数十亿行的巨型仓库）。可行的做法是构造**连贯的序列**：取一个目录里的所有文件，或者按依赖图解析、把相互依赖的文件拼在一起。

一个公开的做法是 Qwen2.5-Coder 的长上下文训练：从 8K 开始训练，再用 YaRN 逐步扩到 128K。长上下文能力是否有效，可以用跨文件补全（cross-file completion）、infilling、短上下文保持等任务来验证。

2.4 本章小结

预训练阶段，代码模型要把代码混进通用语料，处理空白/文件边界/许可证/敏感数据，并做严格去重（fork、vendor、近似重复）；tokenization 要 code-aware、保留空白、压缩高频空白；长上下文要通过目录或依赖图构造连贯序列，并用 YaRN 一类方法扩展。数据量与语言覆盖对代码能力几乎单调有利。

3 中训练：把通用模型改造成代码模型

预训练之后，还有一个常被忽略但极其重要的阶段——**中训练**（mid-training，也常被称为 continued pre-training 或大规模 SFT）。主讲人把它和 SFT 视为同一个阶段。图 **??** 之前的幻灯片给

了两个公开案例：CodeLlama 与 Qwen Coder。

3.1 代码配比：不能只喂代码

CodeLlama 的做法是在 Llama 2 基础上再训练约 5000 亿 token 来适配代码。但这里有一个反直觉的教训：**如果中训练用 100% 代码，代码能力会很好，但数学、问答等通用能力会明显退化**；改成「代码 + 文本 + 数学」的混合后，代码性能只掉一点点，通用能力保住了。原因很现实：现在没有人只用模型写代码，通用能力是产品可用性的一部分。

3.2 Infilling：让模型学会「填中间的空」

普通自回归模型只能看左边的上下文，因此它天然擅长「补全函数后面的代码」，却不擅长「补全中间缺的内容」。但软件开发里大量任务是「中间补全」：在文件中间插一段代码、推断一个返回类型、生成注释、重命名变量。这些任务都需要**同时看前后文**。

图 **??** 的例子是填写返回类型：`def is_positive(x: int) -> ___: return x > 0`。只看左边的自回归模型只能猜（bool? string? enum?），但看到右边的 `return x > 0` 就能确定是 bool。问题是如何训练这种能力。

主讲人引用了一个早期工作（他开玩笑说这是另一位老师 Daniel 的论文，自己是来「剧透」的）：把 infilling 任务**转化成自回归任务**——把代码切成前缀（prefix）、后缀（suffix），中间要填的片段标记为 `M0`；训练时让模型看到「前缀 + M0 + 后缀」的排列，在推理时把 M0 放在前缀与后缀之间，让模型生成 span。这个技巧可以自然扩展到多个洞（M0、M1……依次生成）。

效果如何？主讲人给了数字：在单行与多行补全上，纯从左到右推断大约是 48 到 24、25（不同设置）；做从左到右 + 采样十个候选并用整个文件上下文重排会更好，但仍然显著不如**直接训练 infilling**——而且 infilling 推理不需要任何重排。另一点很关键：训练 infilling 不会牺牲标准的从左到右补全能力（用 causal masking 仍然保持原有性能），所以两者不是取舍关系。

训练细节上，洞的数量与长度用**长尾分布**采样：多数时候一个洞，少数时候两个、三个；长度同理。这个采样分布也可以按真实开发中出现的 infill 模式来设计。

3.3 更多代码数据源：从仓库里「免费」挖数据

除了「代码文本 + infilling」，代码库本身还提供了大量监督信号。主讲人列了几类：

1. **commit diff / PR**：给定 diff 前的文件与 PR 描述，让模型生成 diff 后的结果——这天然是「代码编辑」的训练数据。
2. **诊断/修复**：坏代码 + 错误信息 → 修复。哪里能免费拿到大量这种数据？主讲人的答案是 **CI（持续集成）历史**：公开仓库里记录了「某版本代码跑出错误、后来的版本修好并通过测试」；此外 Stack Overflow、GitHub issue 也是来源。
3. **执行轨迹**：给程序加 instrument、在测试时运行，记录某个时刻的栈、变量、输出——后面「代码世界模型」会用这类数据。

3.4 本章小结

中训练把通用模型改造成代码模型：代码配比不能 100%（会伤通用能力），要代码 + 文本 + 数学混合；infilling（FIM）把「中间补全」转成自回归任务，训练时用 M0/M1 标记洞、推理时按

prefix+hole+suffix 构造，效果明显优于从左到右 + 重排，且不牺牲普通补全；commit diff、CI 失败历史与执行轨迹是免费的高价值数据源。

4 代码评测：从表层相似到执行判定

会写代码之后，下一个问题是「怎么判断写得好不好」。主讲人把评测分成两大范式：**和参考解比较**，以及**真正运行**。

4.1 和参考解比较：BLEU、CodeBLEU 与 CodeBERTScore

最直观的方式是把模型输出与人类参考解比较。早期用 **BLEU** 这类 n-gram 重叠指标——它的问题是：一个 token 之差可能让程序行为完全不同（比如 `>=` 与 `>`），反过来，两个写法差异很大的程序可能完全等价。也就是说，**表层相似度与语义正确性是两回事**。

为缓解这个问题，出现了 **CodeBLEU**：不再比较 token，而是比较程序的**抽象语法树**（AST）结构，以及语法级/语义级的数据流匹配（变量从哪来、到哪去）。另一条路线是 **CodeBERTScore**：把代码的每个 token 做嵌入，再检查一份代码的每个 token 是否能在另一份代码里找到「有对应关系」的 token，从而判断两份实现是否表达同一件事。这些方法比 BLEU 好，但主讲人很快转向今天的主流：执行判定。

4.2 执行判定：跑测试

图 **??** 是执行判定的核心：给候选代码 + 一批测试用例，运行；**全过给 1，否则给 0**。它的好处和局限同样鲜明，幻灯片列得很清楚：

- **好处**：可以接受不同的正确实现（不要求与参考解长得像）；能通过执行发现语义错误；匹配的是可观察行为。
- **局限**：测试有限，可能漏掉错误行为；需要可信的测试、完整的依赖与隔离环境；需要时间与稳定的执行环境。

主讲人展开了两个现实痛点。第一是**测试质量**：几乎每个流行 benchmark 都有论文在批评它的单元测试——要么不够严格导致「错误实现也能过」（假阳性），要么测试要求了题目没说的东西（假阴性）。他举了一个很具体的例子：需求只说「在界面右上角加一个 learn more 按钮」，但测试可能还检查了按钮背景色——这个隐藏要求从一开始就没写在题面里。随着任务越来越难，**同时做到低假阳性和低假阴性**成为训练编程 agent 的最大问题之一。

第二是**环境**：跑测试需要装依赖、需要隔离（常放 Docker 容器），还会遇到版本漂移——依赖升级导致测试挂掉，模型明明没退步却被判低分。大仓库的测试跑一次可能五分钟、十分钟甚至更久，这也是为什么 benchmark 会做「测试子集」。

4.3 单步代码生成的主流 benchmark

图 **??** 是经典中的经典：**HumanEval**，164 个 Python 函数，给 docstring 让模型补全，隐藏单元测试判定。它的优点是简单、便宜；缺点是已经偏旧（几乎所有模型都能拿高分），而且题目是人工构造的「tricky Python 函数」，不完全代表真实开发。

更难的是 **CodeContests / Codeforces** 一类竞赛题：模型要一次性写出能通过全部（含隐藏）测试的程序，对人类来说第一次就 100% 通过也不容易。这类任务对**推理能力**要求极高——这也是代码成为推理模型最成功应用之一的原因。

评测指标上，除了 **pass@1**（只生成一个，对就得分），还有 **pass@k**（生成 k 个里任意一个对就算过）——后者更宽松，通常用于衡量「潜力」而非「一次做对的能力」。

4.4 非功能需求：正确之外的维度

「能跑对」之外，代码还有运行效率、延迟、资源占用、可维护性、安全性等维度。主讲人提到有工作（如 NoPHOTOBELL 一类）不只检查功能正确，还用静态检查评估可维护性与安全性；不过对**非 agent 的单步任务**而言，这些维度大多没有被纳入训练——模型主要被训练成「写得对」，而不是「写得快」。到了 agentic 任务，效率才开始更常被考虑（比如执行超时会直接零分）。

4.5 本章小结

代码评测有两条路：与参考解比较（BLEU 太表层，CodeBLEU 用 AST 与数据流，CodeBERTScore 用嵌入对齐）与执行判定（跑测试，全过为 1）。执行判定是今天的主流，但受限于测试质量（假阳性/假阴性）与执行环境（依赖、版本漂移、时间）。单步 benchmark 以 HumanEval 为代表、CodeContests 更难；指标有 pass@1 与 pass@k；非功能需求在单步训练中通常不被优化。

5 单步代码模型的 RL：从测试结果学习推理

有了执行判定，就可以把它当作奖励来做强化学习。图 **??** 是标准的单步 RL 循环：采样多个回答（问题 → 推理 + 最终代码）→ 运行测试（通过/失败 → 奖励）→ 更新模型（让被奖励的回答更可能）→ 用更新后的模型再采样。

这个循环本身很朴素，但有几点值得强调：

1. **它能训练出强推理模型**。竞赛题一次性做对很难，需要模型「想清楚再写」；而测试结果提供了一个可自动验证的信号，正好适合 RL。主讲人提到一些工作（如 DeepCoder）展示了在中训练之后继续用 RL 还能进一步提升，并且把上下文从 16K 扩到 32K、再扩到 64K 继续训练。

2. **效率是隐式的约束**。很多竞赛题有执行超时；超时直接零分，这相当于给「写得快」提供了一个隐含的奖励信号，而不需要显式定义效率奖励。

3. **注释与非功能质量**。有同学问「为什么模型总爱写一堆注释?」主讲人给了几个假设：训练数据里本来就有注释，模型会模仿；模型被训练成「边写边想」，写注释可以给推理争取更多 token；以及现在有些非功能分类器/代码质量模型会约束注释数量（早期模型曾被抱怨「注释多到没人看」）。类似的，可读性也可以通过 RLHF 的奖励模型（人类标注质量偏好）间接优化。

5.1 本章小结

单步代码模型的 RL 就是「采样 → 跑测试 → 用通过/失败更新模型」的循环；它能把推理能力训进模型（竞赛题受益最大），执行超时提供了隐式的效率约束；注释数量、可读性等非功能维度，主要靠训练数据分布、隐式 token 激励与质量分类器/RLHF 奖励模型来间接控制。

6 Agentic coding：localize–edit–verify 循环

从单步代码生成切换到 agentic coding，最大的变化是：模型开始**迭代地调用工具**，在真实代码库里工作。主讲人先讲最常见的设定——在 GitHub 仓库里修 issue 或加 feature。

6.1 三步循环

图 **??** 把 agentic coding 拆成三步，并循环执行：

1. **Localize（定位）**：找出需要改哪里。例子是「retries 在值为 0 时没有正确取默认值」，要先找到相关实现、复现行为；
2. **Edit（编辑）**：改代码。例子是「只在值为 None 时才设默认值」；
3. **Verify（验证）**：跑测试，确认改对了、没弄坏别的；
4. 如果验证失败，就用失败信息指导下一次迭代——这是 agent 与单步模型最本质的区别。

6.2 一个反例：Agentless 固定工作流

并非所有仓库级任务都必须用 agent 循环。图 **??** 是 Agentless 的固定工作流：先定位到最可能要改的文件（Top-N files），再定位到类与函数（Top-N classes/functions），再定位到具体行，然后让单步语言模型生成 patch，最后过滤与排序、提交。

主讲人很坦诚地点评：这类方法一度「令人沮丧地有效」——因为当时的模型被训练得非常擅长单步解 issue，却不擅长 agentic 的工具调用与验证循环；所以固定工作流反而赢。但现在主流都是用 agent。

6.3 只给一个 bash 工具行不行？

最极简的 agent 是 Mini-SWE-agent 一类：**只给模型一个 bash 工具**。图 **??** 展示了这样一套工具集：读/搜索用 `rg`/`cat`/`find`，编辑用 `sed`/`Python`/`patch`，跑检查用 `pytest`/`build`；模型选择命令，工具返回输出 + 退出码 + 超时。理论上这就足够实现一个编程 agent。

但主讲人解释了为什么实操中还要专门加一个「编辑文件」的工具：用 `sed` 改文件，模型每次都要想「用 sed 还是 Python 还是 patch」，而且转义（反斜杠、特殊字符）很容易出错。如果做一个专门为 agent 设计的文件编辑工具，模型大部分编辑都会用它，正确率会高很多。这也解释了为什么几乎所有有竞争力的编程 agent，都会至少加一个文件编辑工具。

6.4 编辑格式：diff vs 搜索替换

编辑工具的核心是**格式**。图 **??** 对比了两种主流格式：左边是 **unified diff**（带 `@@ -1,2 +1,2 @@` 的行号信息），右边是 **file-operation patch**（Codex 风格，带 `Begin Patch`/`Update File`，**不带行号**）。

主讲人给出一条很实际的教训：让模型维护行号「几乎每次都会错」。因此更常见的是「长得像 diff，但不带行号」的格式——用上下文文本定位要改的地方。它的缺点是：如果同一个文件里有多处 `retries = 3`，模型必须多写几行上下文才能消歧；但这个缺点远小于行号出错的代价，所以几乎所有 agent 都选这种。

这个细节的影响可以大到决定成败。主讲人举了两个例子。一是 **ADE R 代码编辑基准**：它比较 GPT-4 Turbo 在 search-replace 与 unified diff 两种格式上的表现，结果统一 diff 明显更好——因为模型在训练里见过更多这种格式。二是 Gemini：它长期在某些 harness 上表现较差，原因是它被训练的原生编辑格式与其它 harness 不同；直到最近才修好。结论很锋利：**编辑格式不匹配，会成为编程 agent 的巨大瓶颈**。

6.5 代码定位：从 grep 到依赖图

定位阶段最常见的手段是搜索：`ripgrep` 是主流，也有 `grep` 与专门的代码搜索工具；拿到线索后顺着值/引用走。更复杂的一类是 **LocAgent** 这类工作：给 agent 一个能沿依赖路径跳转的工具，找到 `config.py` 之后可以自动跳到相关文件。主讲人的经验很诚实：这类方法**很难打过更简单的做法**（后面他会用实际例子说明）。

6.6 本章小结

agentic coding 的核心是 localize–edit–verify 循环；Agentless 这类固定工作流一度有效，但现在主流是 agent。最简 agent 只需一个 bash 工具，但专用编辑工具能显著提升编辑正确率；编辑格式（行号 diff vs 不带行号的上下文 patch）会影响成败，模型训练格式与 harness 不匹配是真实瓶颈；定位主要靠搜索，依赖图类复杂方法不一定赢过简单搜索。

7 评测与训练数据工程：SWE-bench 及其生态

7.1 SWE-bench 是怎么构造的

编程 agent 最有名的评测是 **SWE-bench**。它的构造方式非常巧妙，值得记住：

1. 从 GitHub issue 出发（这就是题目）；
2. 取该 issue 被解决时的代码库状态（即 PR 之前的版本）作为环境；
3. 找到与该 issue 关联的测试；
4. 让 agent 生成 patch；应用 patch、跑测试；
5. 所有相关测试都通过则得 1，否则 0。

这里的「harness」不是 agent harness，而是**测试 harness**——就是负责跑测试的那层设施。一个重要工程细节是：有些仓库（如 Django、NumPy）跑完整测试套件需要五到十分钟甚至更久，所以 SWE-bench 挑出**必须通过的测试子集**：一类是 **Fail-to-Pass（F2P）**——PR 之前失败、PR 之后通过的测试，正好检验这次修改的功能；另一类是 **Pass-to-Pass（P2P）**——PR 之前就通过、之后也必须通过的测试，用来防止回归。

一个「可运行的训练任务」因此由四部分组成：**起始状态**（文件、依赖）、**测试命令**、**issue（要求的行为）**、**检查（测试）**。

7.2 评测是改进 agent 的最大瓶颈

主讲人给了一个很有洞察力的判断：**评测是今天把 agent 做得更好时最大的问题**。原因有正反两面：一方面，测试质量、环境问题、假阳性/假阴性都很棘手；另一方面，**如果你能大量生成可运行的评测环境，你就可以直接在这上面做 RL**——评测数据就是训练数据。

他介绍了几条代表性工作：

- **SWE-Gym**：为 SWE-bench 额外构造训练环境（SWE-bench 本身只有评测环境），用它们训练模型。他提到一个模型原本只有约 11% 准确率，用 500 个例子训练后就能显著提升；此外还能用「生成多个输出 + 重排」做推理时的扩展。
- **SWE-rebench**：持续不断地从 PR 生成新环境（2024 年 12 月到 2026 年 6 月 30 日的 PR），有训练集与开发集，是目前最大规模的公开编程 agent 数据之一。
- **SWE-smith**：不依赖 PR，而是**注入 bug**：准备可执行的基线代码（现有测试全过），对代码做小变异（比如把 `if value is None` 改成 `if not value: return 3`），保留测试，找出因此失败的测试——这样就有了一个「请修好它」的任务。图 **??** 展示了这条流水线：真实仓库 + 单元测试 → 用 SWE-agent 尝试装环境、让开发者写 Dockerfile → 程序化修改 / LM 生成 / 组合变异 / PR 镜像等策略生成新任务实例 → 带 bug 的 patch 与验证过的测试。优点是**轻量、几乎无限数据**；缺点也很明确：这些是人工注入的 bug，不一定代表真实开发中会遇到的问题。
- **Multi-SWE-bench / SWE-bench Multilingual**：把上面这套扩展到多种编程语言；迁移的主要成本是各语言的项目约定不同，但现在 agent 自己就能处理这些约定。

7.3 多 harness 训练

图 **??** 给出了 harness 的定义：**harness = 提示词 + 工具 + agent loop + 上下文管理**。同一套权重，接上 OpenHands、OpenCode、Codex 等不同 harness，得到的轨迹与任务检查都不同。

正因为「模型训练时的格式」与「harness 期望的格式」往往不一致，现在语言模型厂商常做**多 harness 训练**：不是只在单一 harness 上训练，而是在多个 harness 上一起训。幻灯片给了两个例子：Nemotron 3 Ultra 的每个任务分布用至少 2 个 harness；Polar 则让各家原生 harness 调用一个共享的模型 API 代理（这样模型能见到多种 harness 的轨迹）。

7.4 本章小结

SWE-bench 由「issue + 当时代码 + 相关测试」构造，用 F2P/P2P 测试子集判分；一个可运行任务需要起始状态、测试命令、issue 与检查。评测数据就是训练数据：SWE-Gym 造训练环境、SWE-rebench 持续扩环境、SWE-smith 通过注入 bug 廉价造任务（但不够自然）、Multi-SWE-bench 扩多语言。harness（提示词 + 工具 +loop+ 上下文管理）会显著影响同一模型的表现，因此需要多 harness 训练。

8 不只是修 bug：前端、多语言与更广义的软件开发

8.1 前端开发：需要浏览器验证

前端任务（如 SWE-bench Multimodal）有两个特殊点。第一，输入可能包含图片（issue 里贴的截图），需要多模态理解；第二，验证要**在浏览器里做**——要么用一个 GUI agent 当组件，要么写 **Playwright** 这类脚本做自动化检查。图 **??** 给出了前端 agent 的循环：定位组件与事件处理器 → 修改逻辑与样式 → **在浏览器中验证**（操作行为：输入/点击/检查保存状态；检查渲染页面：截图 → 视觉理解）→ 失败则回头改。

主讲人还讲了一个让他「三观被震」的结果：GLM 5.2 是**非多模态**模型，却拿到前端设计竞技场第一名。他的判断是：多模态对前端 agent 可能仍然重要，但「写得好」与「视觉理解」不是同一件事——模型可以靠代码结构、组件库知识、以及强大的代码先验做得很好。

8.2 内循环与外循环

图 **??** 把软件开发分成内循环（inner loop）与外循环（outer loop）：

- **内循环**：需求 → 实现循环（编辑 → 测试）→ 评审（diff + 证据，release decision）→ 部署/监控（生产信号、回滚路径）→ 维护（新请求、观察到的失败）→ 回到需求。这是本讲前面绝大部分内容所在的循环。
- **外循环**：需求、评审、部署、监控、维护——它们更难定义、也更难找到训练数据，但同样是软件开发的巨大组成部分。

幻灯片还给了两个关键提醒：「测试通过只是一步，不是发布决定」；而 2019 年微软对自家开发者的调查显示，开发者只有约 **15%** 的时间花在写代码（新功能与修 issue）上，约 25% 在会议与邮件、14% 在调试、8% 在跑测试……也就是说，把「写代码」做得再好，也只覆盖了软件工程的一小块。

8.3 任务谱系：修复之外的软件工程

主讲人快速扫了一组「修复 issue 之外」的任务与对应 benchmark：

- **定位（localization）**：专门训练「从哪开始改」。CMU 的 **CodeScout** 用 RL 做这件事，好处是比完整编程 agent 更可控、更适合小模型——主讲人特别建议把它当作课程项目的选题。
- **从零造应用**：**VibeBench** 评测「vibe coding」的能力；难点在于需要好的 verifier。
- **实现完整后端库**：**Commit0** 与 **ProgramBench**；后者要求**逆向工程一个二进制、从零重写实现**——运行二进制、探测输入输出、再重新实现，是当前最长的长程任务之一。
- **软件维护**：**SWE-Milestone**，不是修一个 issue，而是在一个 epoch 里连续实现多个 issue，同时保证代码库不被搞乱、保持可维护性。
- **测试生成**：**SWT-Bench**，用变异测试的思路验证「你写的测试能不能抓住 bug」；主讲人强调，能写出好测试来验证 issue 真的被修好，本身就解决了问题的一半。
- **部署/运维**：JetBrains 有相关 benchmark 尝试，但这类任务需要真实基础设施，目前还没有很好的公开评测。

8.4 本章小结

前端 agent 需要多模态 + 浏览器验证（GUI agent 或 Playwright），但非多模态模型也能在前端设计上表现很好；软件工程分内循环（编辑 测试）与外循环（需求/评审/部署/监控/维护），开发者只有约 15% 时间在写代码；任务谱系远超「修 issue」：定位（CodeScout）、造应用（VibeBench）、库实现（Commit0/ProgramBench）、维护（SWE-Milestone）、测试生成（SWT-Bench）与部署等。

9 代码世界模型与训练迁移

9.1 预测执行：让模型「在脑子里运行程序」

主讲人最后回到一个很小众但很酷的方向：**代码世界模型**（code world model）——让模型预测程序的执行结果，而不是真的运行。例子是 Python 的别名/引用行为：给定一段涉及 `items = []`、`alias = items`、`items.append(2)` 的代码，问「items 和 result 分别是什么」。**把别名误当成拷贝**是很常见的错误；而如果模型能准确预测执行状态，它就能在不执行的情况下做调试、规划与推理。

这类能力可以靠**执行轨迹**数据训练：给程序加 instrument、运行、记录状态，然后让模型从代码预测状态转移。

9.2 训练能迁移到新的开发任务

图 **??** 给了最有力的证据：在 Hybrid-Gym 上训练的 32B 模型（红）与 Base 32B（蓝）相比，在三种不同的开发任务上都有明显提升——**SWE-bench Verified（修复）7 → 32.4**、**SWT-Bench Verified（生成测试）9.01 → 16.86**、**Commit0 Lite（实现库）8.34 → 13.45**。这说明在「代码行为/开发环境」上学到的东西，可以迁移到不同的开发任务上，而不是只对训练分布有效。

9.3 本章小结

代码世界模型让模型预测程序执行状态（如别名/引用），可支撑不执行情况下的调试与规划，训练数据来自执行轨迹；Hybrid-Gym 的实验表明，在代码行为与开发环境上的训练可以迁移到修复、测试生成、库实现等多种新任务。

10 总结与延伸

10.1 主讲人的收尾

主讲人最后用一页「设计一个编程 agent」的清单收束全讲（因时间关系只讲到这里，代码世界模型部分留到答疑）：

设计编程 agent 的检查清单（据主讲人收尾整理）

1. **代码模型**：预训练、中训练、以及从执行奖励出发的 RL；
2. **agentic coding**：用合适的工具做 localize、edit、verify；
3. **agent 训练**：可运行任务、修复奖励、多样 harness；
4. **前端开发**：浏览器交互与视觉验证；
5. **更广义的开发**：定位、应用生成、库实现、维护、测试与 CI；
6. **代码行为预测**：用学到的世界模型支撑执行与规划。

10.2 把本讲压缩成六句话

本讲核心结论（背这几句）

1. 代码模型的地基是预训练（代码混语料 + 严格清洗 + code-aware tokenization + 长上下文）与中训练（代码配比、infilling、更多数据源）。
2. 代码评测从「和参考解比」走向「执行判定」；执行判定是主流，但测试质量与执行环境是天花板。
3. 单步代码模型的 RL = 采样 + 跑测试 + 用通过/失败更新；它能训出推理能力，执行超时提供隐式效率约束。
4. agentic coding 的骨架是 localize–edit–verify 循环；工具与**编辑格式**是能力的一部分，不同 harness 之间差异巨大。
5. SWE-bench 生态说明「评测数据就是训练数据」: F2P/P2P、SWE-Gym、SWE-rebench、SWE-smith、多 harness 训练。
6. 编程 agent 的边界正在从「修 issue」扩到前端、定位、库实现、维护、测试与部署；代码世界模型让模型能预测执行，训练可以迁移到新任务。

10.3 延伸思考

1. **harness 是模型的「第二外壳」**。上一讲讲计划与多 agent，这一讲把 harness 拆成提示词、工具、agent loop、上下文管理四件事。同一套权重换 harness 分数就变——这解释了为什么厂商要做多 harness 训练，也解释了为什么「适配某家 harness」本身是工程护城河。
2. **编辑格式是最被低估的工程变量**。行号 diff 几乎必错、上下文 patch 更稳；模型在原格式上训过就强、没训过就弱。做 coding agent 时，第一件该对齐的事情不是 prompt，而是**编辑格式与工具返回格式**。
3. **测试质量决定训练上限**。假阳性让模型学会「糊弄」，假阴性让好模型被误判；F2P/P2P、mutation testing（SWE-smith/SWT-Bench）本质上都是在解决「怎么让验证信号可信」。测试写得好，问题就解决了一半。

4. **评测环境即训练环境**。SWE-Gym、SWE-rebench、注入 bug 的共同逻辑是：把评测环境变成可以无限生成的训练环境。这与上一讲「计划需要重规划」一样，都是把「评估」从终点变成数据来源。

5. **从修 bug 到软件工程全流程**。15% 的时间在写代码，剩下 85% 在需求、评审、部署、监控、维护。下一阶段的机会很可能在「外循环」，而不是把 SWE-bench 分数再刷高几个点。

10.4 拓展阅读（课程中提到的文献，供进一步查阅）

- StarCoder / StarCoderBase、CodeLlama、Qwen2.5-Coder：代码模型预训练与中训练的代表工作。
- Infilling（FIM）方向：把中间补全转为自回归任务（讲座提到的工作与 Daniel 相关）。
- CodeBLEU、CodeBERTScore：代码相似度评测；HumanEval、CodeContests：单步代码基准。
- DeepCoder：用 RL 与长上下文训练代码推理模型。
- Agentless（CMU 工作）与 Mini-SWE-agent：固定工作流与 shell-only agent 两条路线。
- SWE-bench、SWE-Gym、SWE-rebench、SWE-smith、Multi-SWE-bench：评测/训练环境生态。
- LocAgent、CodeScout：代码定位；VibeBench、Commit0、ProgramBench、SWT-Bench、SWE-Milestone：修复之外的任务基准。
- Nemotron（NVIDIA 技术报告）的多 harness 训练；Hybrid-Gym / SWE-Playground（CMU）的自动任务生成。